TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI TP.HCM
VIỆN CÔNG NGHỆ THÔNG TIN, ĐIỆN, ĐIỆN TỬ



Chapter 3.

TRAINING DEEP NEURAL NETWORKS

PhD. Nguyễn Thị Khánh Tiên
tiennntk@ut.edu.vn

Đào tạo mạng lưới thần kinh sâu

Mạng nơ-ron sâu (DNN) là các mô hình máy tính được xây dựng từ nhiều lớp nơ-ron nhân tạo, mô phỏng cách bộ não con người xử lý thông tin. "Sâu" dùng để chỉ số lượng lớp ẩn trong mạng. Mạng có nhiều lớp ẩn hơn có thể học các mẫu phức tạp hơn từ dữ liệu.

Đào tạo DNN là quá trình dạy một mô hình máy tính thực hiện một nhiệm vụ cụ thể bằng cách sử dụng dữ liệu.

Quy trình đào tạo DNN:

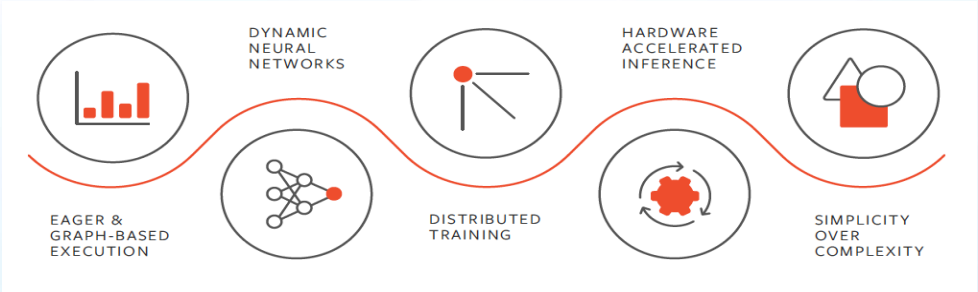
- Dữ liệu: Cần một lượng lớn dữ liệu được dán nhãn để huấn luyện mô hình.
- Thuật toán: Thuật toán phổ biến nhất để đào tạo DNN là lan truyền ngược. Thuật toán này điều chỉnh trọng số của mạng để giảm thiểu sai số giữa đầu ra dự đoán và đầu ra thực tế.
- Tối ưu hóa: Các thuật toán tối ưu hóa (ví dụ: Adam, SGD) được sử dụng để tìm trọng số tối ưu.
- Lặp lại: Quá trình huấn luyện thường được lặp lại nhiều lần (epoch) để mô hình học các mẫu chính xác hơn.

Thách thức trong việc đào tạo DNN:

- Trang bị quá mức: Mô hình học dữ liệu huấn luyện quá tốt nhưng hoạt động kém trên dữ liệu mới.
- Độ dốc biến mất/bùng nổ: Các vấn đề xảy ra khi độ dốc trở nên quá nhỏ hoặc quá lớn trong quá trình truyền ngược, khiến việc huấn luyện trở nên khó khăn.
- Yêu cầu tính toán cao: Việc đào tạo DNN đòi hỏi sức mạnh tính toán đáng kể, đặc biệt đối với các mạng lớn và bộ dữ liệu lớn.

Công cụ và thư viện:

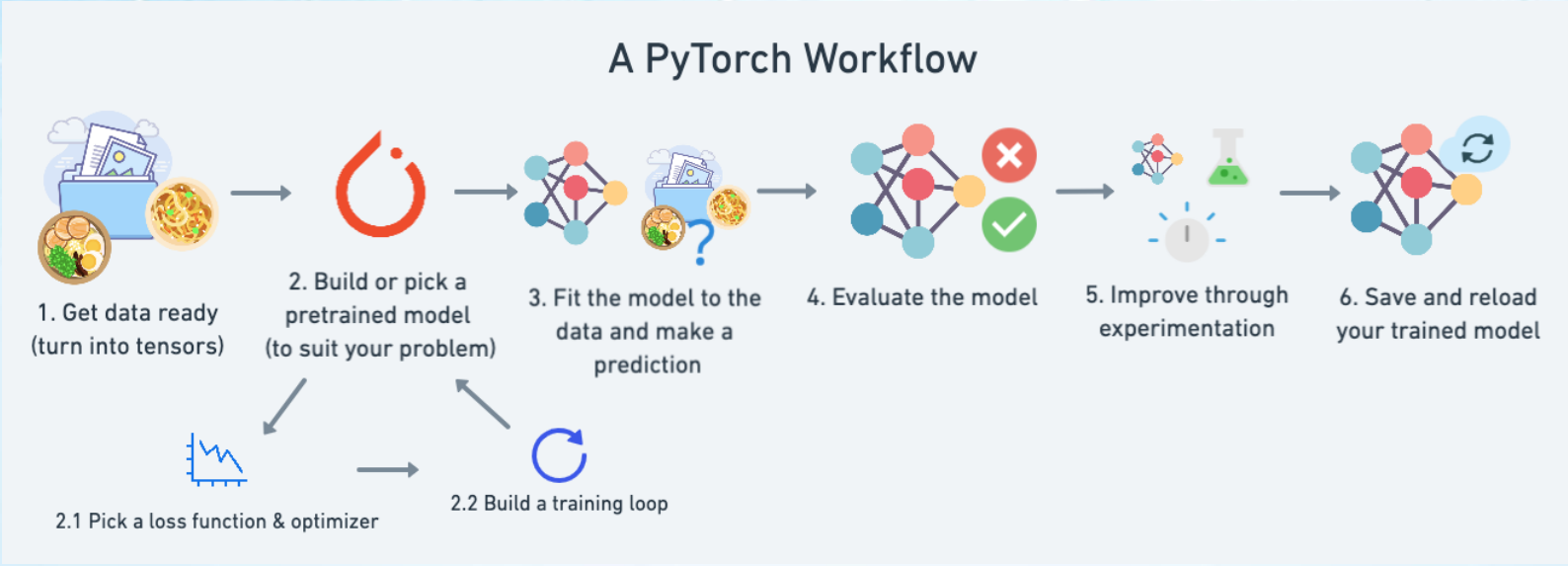
- TenorFlow
- **PyTorch**
- máy ảnh



Đào tạo DNN bằng PyTorch

Các bước cơ bản Các bước đào tạo DNN bằng PyTorch:

1. Chuẩn bị dữ liệu của bạn (Bộ dữ liệu và Trình tải dữ liệu) 2. Xác định mô hình của bạn (Kiến trúc mạng thần kinh) 3. Chọn hàm mất dữ liệu và trình tối ưu hóa
4. Viết vòng lặp đào tạo
5. Xác thực (Tùy chọn nhưng được khuyến nghị cao) 6. Lưu và tải mô hình của bạn (để sử dụng sau)



Chuẩn bị dữ liệu (Bộ dữ liệu và Trình tải dữ liệu)

PyTorch cung cấp các lớp **Bộ dữ liệu** và **DataLoader** để quản lý dữ liệu của bạn một cách hiệu quả.

- **Bộ dữ liệu:** Lớp này chịu trách nhiệm:
  - Lưu trữ các mẫu và nhãn tương ứng của chúng.
  - Đang triển khai **__len__** (trả về kích thước của tập dữ liệu).
  - Đang triển khai **__getitem__** (trả về một mẫu và nhãn của nó tại một chỉ mục nhất định).

- Bạn thường sẽ tạo một lớp **Bộ dữ liệu** tùy chỉnh kế thừa từ **torch.utils.data.Dataset.** Bạn sẽ cần điều chỉnh điều này cho phù hợp với định dạng dữ liệu cụ thể của mình.

- **DataLoader**: Lớp này chịu trách nhiệm:
  - Lặp lại thông qua tập dữ liệu theo đợt.
  - Xáo trộn dữ liệu (quan trọng cho việc đào tạo).
  - Tải dữ liệu song song bằng nhiều công nhân (tăng tốc độ tải dữ liệu).

```python
import torch
from torch.utils.data import Dataset, DataLoader

class SimpleDataset(Dataset):
    def __init__(self, data, labels):
        self.data = data
        self.labels = labels

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        sample = self.data[idx]
        label = self.labels[idx]
        return sample, label

# Example Usage:
data = torch.randn(100, 10) # 100 samples, each 10 features
labels = torch.randint(0, 2, (100,)) # Binary classification labels (0 or 1)

dataset = SimpleDataset(data, labels)
dataloader = DataLoader(dataset, batch_size=32, shuffle=True)

```

Chuẩn bị dữ liệu (Bộ dữ liệu và Trình tải dữ liệu)

Ví dụ: Đối với các tập dữ liệu phổ biến (như MNIST, CIFAR10, v.v.), mô-đun PyTorch `torchvision.datasets` cung cấp các tập dữ liệu dựng sẵn. Bạn có thể tải xuống và sử dụng chúng trực tiếp.

```python
from torchvision import datasets, transforms

# Define transformations to preprocess the data (e.g., normalization)
transform = transforms.Compose([
    transforms.ToTensor(), # Convert PIL image to tensor
    transforms.Normalize((0.1307,), (0.3081,)) # Normalize for MNIST
])

# Load MNIST dataset
train_dataset = datasets.MNIST(root='./data', train=True, download=True, transform=transform)
train_dataloader = DataLoader(train_dataset, batch_size=64, shuffle=True)

test_dataset = datasets.MNIST(root='./data', train=False, download=True, transform=transform)
test_dataloader = DataLoader(test_dataset, batch_size=1000, shuffle=False) # No need to shuffle test data
```

Xác định mô hình (Kiến trúc mạng thần kinh)

Bạn sẽ sử dụng **torch.nn.Module** để xác định kiến trúc DNN của mình. Điều này liên quan đến việc tạo các lớp kế thừa từ **nn.Module** và xác định:

- **__init__(self):** Nơi bạn xác định các lớp mạng của mình (ví dụ: **nn.Linear, nn.Conv2d, nn.ReLU, nn.MaxPool2d**, v.v.).
- **forward(self, x)**: Nơi bạn xác định đường chuyển tiếp của mạng, chỉ định cách dữ liệu truyền qua các lớp. **Ví dụ:**

DNN Feedforward đơn giản để phân loại

```python
import torch.nn as nn
import torch.nn.functional as F # Functional interface for activation functions

class SimpleDNN(nn.Module):
    def __init__(self, input_size, hidden_size, num_classes):
        super(SimpleDNN, self).__init__()
        self.fc1 = nn.Linear(input_size, hidden_size) # First fully connected layer
        self.relu = nn.ReLU() # ReLU activation function
        self.fc2 = nn.Linear(hidden_size, num_classes) # Output layer

    def forward(self, x):
        out = self.fc1(x)
        out = self.relu(out)
        out = self.fc2(out)
        return out

# Example instantiation:
input_size = 10 # Example input feature size
hidden_size = 50
num_classes = 2 # Binary classification
model = SimpleDNN(input_size, hidden_size, num_classes)

```

Chọn hàm mất và trình tối ưu hóa

Hàm tổn thất (Tiêu chí): Đo lường mức độ hoạt động của mô hình của bạn. Bạn cần chọn một hàm mất thích hợp cho nhiệm vụ của mình (phân loại, hồi quy, v.v.). Các tùy chọn phổ biến bao gồm:

- **nn.CrossEntropyLoss** (để phân loại nhiều lớp)
- **nn.BCELoss** hoặc **nn.BCEWithLogitsLoss** (để phân loại nhị phân)
- **nn.MSELoss** (đối với hồi quy)

Trình tối ưu hóa: Thuật toán cập nhật các tham số của mô hình (trọng số và độ lệch) để giảm thiểu hàm mất mát. phổ biến trình tối ưu hóa:

- **torch.optim.SGD** (Giảm độ dốc ngẫu nhiên)
- **torch.optim.Adam** (Trình tối ưu hóa Adam - thường là một mặc định tốt)
- **torch.optim.RMSprop**

```
import torch.optim as optim

# Loss function for multi-class classification (if num_classes > 2) or binary (if num_classes == 2)
criterion = nn.CrossEntropyLoss() # Or nn.BCEWithLogitsLoss if binary with sigmoid output

# Optimizer (Adam is a good general choice)
optimizer = optim.Adam(model.parameters(), lr=0.001) # lr is the learning rate
```

Write the Training Loop

It typically involves iterating over your data in epochs and batches.
Basic Training Loop Structure:

```
num_epochs = 10 # Example number of epochs

for epoch in range(num_epochs):
    for batch_idx, (data, labels) in enumerate(train_dataloader):
        # 1. Move data to device (GPU if available, otherwise CPU)
        data = data # .to(device) if using GPU
        labels = labels # .to(device)

        # 2. Zero gradients (important before each batch)
        optimizer.zero_grad()

        # 3. Forward pass: compute model outputs
        outputs = model(data)

        # 4. Calculate loss
        loss = criterion(outputs, labels)

        # 5. Backward pass: compute gradients
        loss.backward()

        # 6. Optimization step: update weights
        optimizer.step()

        # Print training statistics (optional, but good for monitoring)
        if (batch_idx+1) % 10 == 0: # Print every 10 batches
            print(f'Epoch: {epoch+1}/{num_epochs}]')
            print(f'Batch [{batch_idx+1}/{len(train_dataloader)}], Loss: {loss.item():.4f}')

print("Training finished!")
```

Giải thích về các bước vòng lặp đào tạo

1. Di chuyển dữ liệu sang thiết bị: Nếu bạn đang sử dụng GPU để đào tạo nhanh hơn (rất khuyến khích cho DNN), bạn cần di chuyển dữ liệu và mô hình của mình sang thiết bị GPU. Bạn sẽ sử dụng **.to(device) f**hoặc thiết bị này (trong đó **thiết bị** là **'cuda'** nếu CUDA có sẵn hoặc **'cpu').**

2. Độ chuyển màu bằng 0: Optimizer.zero_grad() xóa độ chuyển màu khỏi lô trước đó. PyTorch tích lũy gradient theo mặc định, vì vậy bạn phải đưa chúng về 0 trước mỗi đợt mới.

3. Chuyển tiếp: đầu ra = mô hình (dữ liệu) cung cấp dữ liệu đầu vào **(dữ liệu)** thông qua mô hình của bạn để nhận được dự đoán của mô hình **(đầu ra).**

4. Tính tổn thất: tổn thất = tiêu chí (đầu ra, nhãn) ctính toán tổn thất giữa đầu ra của mô hình **(đầu ra) và**và nhãn thực **(nhãn)** sử dụng hàm mất mát đã chọn của bạn.

5. Backward Pass: loss.backward() tính toán độ dốc của tổn thất đối với tất cả các tham số của mô hình (trọng số và độ lệch). Đây là bước lan truyền ngược.

6. Bước Tối ưu hóa: Optimizer.step() cập nhật các tham số của mô hình dựa trên độ dốc được tính toán và thuật toán tối ưu hóa mà bạn đã chọn (ví dụ: Adam).

Xác thực

Điều quan trọng là đánh giá hiệu suất mô hình của bạn trên một tập dữ liệu xác thực riêng trong quá trình đào tạo. Giám sát tình trạng trang bị quá mức (khi mô hình hoạt động tốt trên dữ liệu huấn luyện nhưng kém trên dữ liệu không nhìn thấy).

- Chọn mô hình tốt nhất trong quá trình đào tạo (ví dụ: dựa trên độ chính xác xác thực).
- Điều chỉnh các siêu tham số (tốc độ học tập, chính quy hóa, v.v.).

Những điểm chính để xác thực:

- **model.eval():** Đưa mô hình vào chế độ đánh giá. Điều này rất cần thiết cho các lớp như Chuẩn hóa hàng loạt và Bỏ học, hoạt động khác nhau trong quá trình đào tạo và đánh giá.
- **torch.no_grad():** Tắt tính toán độ dốc, giúp xác thực nhanh hơn và sử dụng ít bộ nhớ hơn.
- **model.train():** Sau khi xác thực, chuyển mô hình trở lại chế độ huấn luyện cho giai đoạn huấn luyện tiếp theo.

Xác thực

Ví dụ: Vòng xác thực (sẽ được thêm vào trong vòng đào tạo):

```python
# ... inside the training loop, after the epochs loop

def validate(model, dataloader, criterion, device):
    model.eval() # Set model to evaluation mode (important for batch normalization, dropout, etc.)
    total_loss = 0
    correct_predictions = 0
    total_samples = 0

    with torch.no_grad(): # Disable gradient calculations during validation (for efficiency)
        for data, labels in dataloader:
            data = data.to(device)
            labels = labels.to(device)
            outputs = model(data)
            loss = criterion(outputs, labels)
            total_loss += loss.item()

            _, predicted_labels = torch.max(outputs.data, 1) # For classification, get predicted class indices
            total_samples += labels.size(0)
            correct_predictions += (predicted_labels == labels).sum().item()

    avg_loss = total_loss / len(dataloader)
    accuracy = 100 * correct_predictions / total_samples
    return avg_loss, accuracy

# ... Inside the epochs loop, after each epoch (or after certain intervals)

    # After each epoch (or less frequently):
    val_loss, val_accuracy = validate(model, test_dataloader, criterion, device) # Using test_dataloader as validation set here for simplicity
    print(f'Epoch [{epoch+1}/{num_epochs}], Validation Loss: {val_loss:.4f}, Validation Accuracy: {val_accuracy:.2f}%')

    model.train() # Set model back to training mode after validation
```

Lưu và tải mô hình của bạn

Sau khi đào tạo, bạn sẽ muốn **lưu các tham số đã học của mô hình (trọng số và độ lệch)** để bạn có thể sử dụng lại sau này mà không cần đào tạo lại

```
torch.save(model.state_dict(), 'model.pth') # Save only the model's parameters
# Or to save the entire model structure (less common for deployment)
torch.save(model, 'full_model.pth')
```

Đang tải mô hình

```
# 1. Instantiate your model class (structure must be the same as when you saved it)
loaded_model = SimpleDNN(input_size, hidden_size, num_classes)

# 2. Load the saved state dictionary
loaded_model.load_state_dict(torch.load('model.pth'))

# 3. Put the model in evaluation mode if you're only doing inference
loaded_model.eval()
```

Trực quan hóa mô hình, dữ liệu và đào tạo bằng TensorBoard

TensorBoard là một công cụ mạnh mẽ để trực quan hóa các khía cạnh khác nhau của mô hình, dữ liệu và quy trình đào tạo PyTorch của bạn.

Hướng dẫn →https://pytorch.org/tutorials/intermediate/tensorboard_tutorial.html

Ưu điểm:

- Trực quan hóa thời gian thực: Xem tiến trình đào tạo của bạn khi nó diễn ra.
- Gỡ lỗi dễ dàng: Xác định các vấn đề như biến mất độ dốc hoặc trang bị quá mức.
- So sánh mô hình: So sánh các mô hình hoặc siêu tham số khác nhau.
- Khám phá dữ liệu: Hiểu dữ liệu của bạn thông qua trực quan hóa.
- Chia sẻ kết quả: Dễ dàng chia sẻ tiến độ luyện tập của bạn với người khác.

1. Cài đặt

- **Đảm bảo bạn đã cài đặt tensorboard và torchvision:** ***pip install tensorboard torchvision***

2. Thiết lập TensorBoard trong PyTorch

- Nhập ***SummaryWriter***: Lớp này chịu trách nhiệm ghi dữ liệu vào nhật ký TensorBoard. ***từ torch.utils.tensorboard import SummaryWriter***
- Tạo một phiên bản ***SummaryWriter***: Chỉ định thư mục nơi bạn muốn lưu trữ nhật ký. ***writer = SummaryWriter('runs/experiment_1') # 'runs/experiment_1' là thư mục nhật ký***

Trực quan hóa mô hình, dữ liệu và đào tạo bằng TensorBoard

3. Trực quan hóa các đại lượng vô hướng (Mất mát, Độ chính xác, v.v.)

Thêm các giá trị vô hướng: Sử dụng **writer.add_scalar() để** ghi lại các giá trị vô hướng như độ mất mát, độ chính xác hoặc tốc độ học tập ở mỗi bước đào tạo.

4. Trực quan hóa Kiến trúc Mô hình

Thêm biểu đồ: Sử dụng **writer.add_graph()** để trực quan hóa cấu trúc mô hình của bạn. Bạn cần cung cấp đầu vào giả để theo dõi mô hình.

5. Trực quan hóa hình ảnh *Thêm hình ảnh:* Sử dụng **writer.add_image()** để trực quan hóa hình ảnh từ tập dữ liệu hoặc dự đoán của bạn.

6. Trực quan hóa biểu đồ

Thêm biểu đồ: Sử dụng **writer.add_histogram()** để trực quan hóa sự phân bổ trọng số hoặc kích hoạt.

7. Trực quan hóa phần nhúng

Thêm nội dung nhúng: Sử dụng **writer.add_embedding()** để trực quan hóa dữ liệu nhiều chiều (ví dụ: nội dung nhúng từ) trong không gian có chiều thấp hơn.

8. Chạy TensorBoard *Mở TensorBoard:* Trong thiết bị đầu cuối của bạn, điều hướng đến thư mục chứa nhật ký của bạn và chạy: *Bash **tensorboard --logdir=chạy Access TensorBoard:*** Mở trình duyệt web của bạn và truy cập **http://localhost:6006/.**

Visualizing Models, Data, and Training with TensorBoard

```python
import torch
import torch.nn as nn
import torchvision.models as models
▶ Launch TensorBoard Session
from torch.utils.tensorboard import SummaryWriter
import torchvision.transforms as transforms
import torchvision.datasets as datasets
import torchvision.utils as vutils

# Create a SummaryWriter
writer = SummaryWriter('runs/resnet18_experiment')

# Load a pre-trained model
model = models.resnet18(pretrained=False)

# Create a dummy input
dummy_input = torch.rand(1, 3, 224, 224)
writer.add_graph(model, dummy_input)

```

```python

# Create example data
transform = transforms.Compose([transforms.ToTensor()])
dataset = datasets.MNIST(root='./data', train=True, download=True, transform=transform)
dataloader = torch.utils.data.DataLoader(dataset, batch_size=64)

# Example training loop (simplified)
for epoch in range(2):
    for i, (images, labels) in enumerate(dataloader):
        # ... (Training code) ...
        loss = torch.rand(1) # example loss.
        writer.add_scalar('Loss/train', loss, epoch * len(dataloader) + i)
        if i % 100 == 0:
            grid = vutils.make_grid(images)
            writer.add_image('MNIST_images', grid, epoch * len(dataloader) + i)

for name, param in model.named_parameters():
    writer.add_histogram(name, param, 1)

# Close the writer
writer.close()

#Run in terminal: tensorboard --logdir=runs
```

Độ dốc biến mất và bùng nổ

Độ dốc biến mất:

- Xảy ra khi độ dốc trở nên quá nhỏ trong quá trình truyền ngược, khiến trọng số ở các lớp đầu cập nhật không hiệu quả.
- Phổ biến với các hàm kích hoạt như sigmoid hoặc tanh.

Độ dốc bùng nổ:

- Xảy ra khi gradient trở nên quá lớn, dẫn đến cập nhật trọng số không ổn định và có khả năng làm gián đoạn quá trình huấn luyện.
- Phổ biến trong các mạng sâu hoặc mạng thần kinh tái phát (RNN).

Giải pháps:

- Sử dụng các chức năng kích hoạt như ReLU hoặc các biến thể của nó (ví dụ: LeakyReLU) để giảm thiểu độ dốc biến mất.
- Áp dụng cắt độ dốc để hạn chế độ dốc quá lớn.
- Khởi tạo trọng số cẩn thận (sẽ đề cập sau).
- sử dụng chuẩn hóa hàng loạt.

Khởi tạo trọng số

Mục đích: Khởi tạo mức tạ phù hợp để thúc đẩy quá trình tập luyện ổn định và hiệu quả.

Tôi chung thods:

- *Khởi tạo ngẫu nhiên:* Khởi tạo trọng số từ phân phối ngẫu nhiên (ví dụ: chuẩn hoặc đồng nhất).
- *Khởi tạo Xavier/Glorot:* Khởi tạo trọng số dựa trên số lượng đầu vào và đầu ra của một lớp, cân bằng độ dốc trong quá trình truyền ngược.
- *He Khởi tạo:* Tương tự như Xavier, nhưng được tối ưu hóa cho các chức năng kích hoạt ReLU.

Triển khai trong PyTorch:

- Sử dụng **torch.nn.init** to khởi tạo trọng số.

```
1  import torch.nn as nn
2  import torch.nn.init as init
3
4  linear_layer = nn.Linear(10, 5)
5  init.xavier_uniform_(linear_layer.weight) #Xavier init
```

Chức năng kích hoạt & Chuẩn hóa hàng loạt

Chức năng kích hoạt

- **Mục đích:** Đưa tính phi tuyến tính vào mạng lưới thần kinh, cho phép chúng học các kiến thức phức tạp các mối quan hệ.
- **Chức năng kích hoạt chung:**
  - **ReLU (Đơn vị tuyến tính chỉnh lưu): ReLU(x) = max(0, x).** Đơn giản, hiệu quả và giảm thiểu độ dốc biến mất.
  - **Sigmoid:**giá trị từ 0 đến 1, thường được sử dụng trong phân loại nhị phân.
  - **Tanh (Tang Hyperbolic):** giá trị trong khoảng từ -1 đến 1.
  - **LeakyReLU:** cải thiện ReLU bằng cách cho phép một gradient nhỏ khi đầu vào âm.
- **Triển khai trong PyTorch:** Sử dụng các lớp trong **torch.nn (**ví dụ**, nn.ReLU, nn.Sigmoid, nn.Tanh, nn.LeakyReLU).**

```python
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(784, 128),
    nn.BatchNorm1d(128),
    nn.ReLU(),
    nn.Linear(128, 10)
)
```

Bình thường hóa hàng loạt

- **Mục đích:** Bình thường hóa đầu ra của các lớp trong mạng để giảm thiểu độ dốc biến mất và bùng nổ, đồng thời tăng tốc quá trình đào tạo.
- **Cách thức hoạt động:**
  - Chuẩn hóa các đầu ra của lớp thành phân phối chuẩn (trung bình = 0, phương sai = 1).
  - Áp dụng các tham số tỷ lệ và dịch chuyển để cho phép mạng tìm hiểu phân phối tối ưu.
- **Triển khai trong PyTorch:**
  - Sử dụng **nn.BatchNorm1d** (đối với dữ liệu 1D)**, nn.BatchNorm2d** (đối với dữ liệu 2D), hoặc **nn.BatchNorm3d** (đối với dữ liệu 3D).

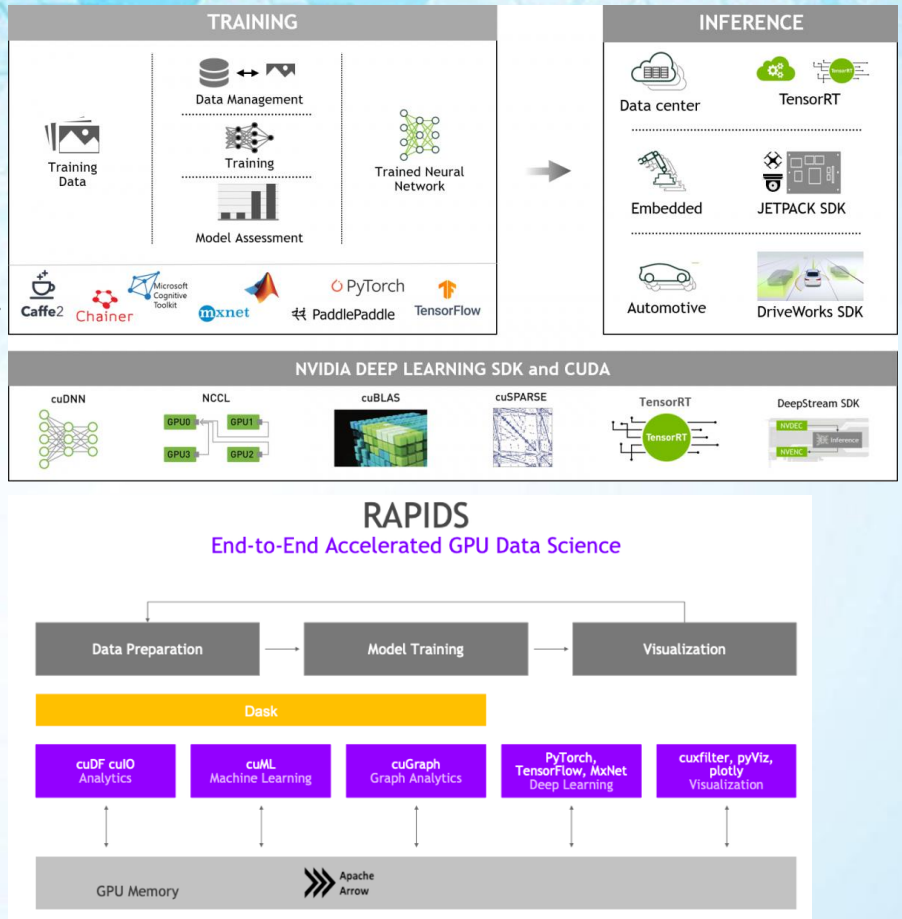
Những cân nhắc quan trọng:

Thiết bị (CPU so với GPU): Đối với đào tạo DNN trong thế giới thực, việc sử dụng GPU rất quan trọng đối với tốc độ. Bạn sẽ cần di chuyển dữ liệu và mô hình của mình sang thiết bị GPU bằng cách sử dụng **.to(device)** where **thiết bị = torch.device('cuda' if torch.cuda.is_available() else 'cpu').**

Điều chỉnh siêu tham số: Tốc độ học, kích thước lô, số kỷ nguyên, kiến trúc mạng (số lớp, đơn vị ẩn), tham số chính quy hóa, v.v., là các siêu tham số. Thông thường, bạn sẽ cần phải thử nghiệm và điều chỉnh những giá trị này để tìm ra giá trị tốt nhất cho vấn đề cụ thể của bạn.

Các mô hình phức tạp hơn: Đối với dữ liệu hình ảnh, bạn sẽ sử dụng Mạng thần kinh chuyển đổi (CNN) với các lớp như **nn.Conv2d, nn.MaxPool2d,**, v.v. Đối với dữ liệu chuỗi (văn bản, chuỗi thời gian), bạn có thể sử dụng Mạng thần kinh tái phát (RNN) hoặc Máy biến áp.

Tham khảo → https://www.nvidia.com/en-au/glossary/pytorch/



Sử dụng lại các lớp được đào tạo trước (Học chuyển giao)

Mục đích: Để tận dụng kiến thức từ các mô hình được đào tạo trước trên các tập dữ liệu lớn (ví dụ: ImageNet) để cải thiện hiệu suất trên tập dữ liệu nhỏ hơn của bạn.

Steps:

1. Tải Mô hình được đào tạo trước: PyTorch cung cấp nhiều mô hình được đào tạo trước trong **torchvision.models.**

1. Đóng băng các lớp: Đóng băng các lớp của mô hình được đào tạo trước để ngăn chúng cập nhật trọng số trong quá trình đào tạo lại.

1. Thay thế hoặc Thêm lớp: Thay thế lớp phân loại cuối cùng của mô hình để khớp với số lượng lớp trong tập dữ liệu của bạn. Bạn cũng có thể thêm các lớp bổ sung để tùy chỉnh.

1. Đào tạo lại: Đào tạo lại mô hình trên tập dữ liệu của bạn. Chỉ trọng số của các lớp chưa đông cứng mới được cập nhật.

```python
import torch
import torchvision.models as models
import torch.nn as nn

# Load pre-trained ResNet-18
model = models.resnet18(pretrained=True)

# Freeze all parameters in the model
for param in model.parameters():
    param.requires_grad = False

# Replace the final fully connected layer
num_ftrs = model.fc.in_features
model.fc = nn.Linear(num_ftrs, 10)  # 10 is your number of classes

# Retrain the model
# ... (training code)
```

Tinh chỉnh

Mục đích: Để tinh chỉnh trọng số của mô hình được đào tạo trước trên tập dữ liệu của bạn nhằm cải thiện hiệu suất.

Steps:

1. Load Pre-training Model: Tương tự như transfer learning.

2. Hủy cố định các lớp: Hủy cố định một số lớp cuối cùng của mô hình để cho phép chúng cập nhật trọng số của mình. *3. Đào tạo lại:* Đào tạo lại mô hình trên tập dữ liệu của bạn.

Học chuyển và tinh chỉnh đặc biệt hữu ích khi bạn có một tập dữ liệu nhỏ.

```python
import torch
import torchvision.models as models
import torch.nn as nn

# Load pre-trained ResNet-18
model = models.resnet18(pretrained=True)

# Unfreeze some of the final layers
for param in model.layer4.parameters():
    param.requires_grad = True

# Replace the final fully connected layer
num_ftrs = model.fc.in_features
model.fc = nn.Linear(num_ftrs, 10)

# Retrain the model
# ... (training code)
```

Kỹ thuật chính quy hóa

Mục đích: Để ngăn chặn việc trang bị quá mức bằng cách thêm một hình phạt vào hàm mất mát. Thử nghiệm với các siêu tham số khác nhau (ví dụ: tốc độ học tập, tỷ lệ bỏ học, cường độ chính quy hóa) để tìm ra các giá trị tốt nhất cho tập dữ liệu của bạn.

Đropout

Mục đích: Vô hiệu hóa ngẫu nhiên các nơ-ron trong quá trình huấn luyện để ngăn chặn sự đồng thích ứng của các nơ-ron. *Triển khai trong PyTorch:* Sử dụng lớp **nn.Dropout** để thêm các lớp bỏ học vào mô hình của bạn. Bỏ học chỉ được áp dụng trong quá trình đào tạo.

```python
import torch.nn as nn
import torch.optim as optim

# Model with dropout
model = nn.Sequential(
    nn.Linear(784, 128),
    nn.ReLU(),
    nn.Dropout(0.5),  # Dropout with probability 0.5
    nn.Linear(128, 10)
)

# Optimizer with L2 regularization
optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=1e-5)
```

L1 và Chính quy hóa L2:

L1 Chính quy hóa (Lasso): Cộng tổng các giá trị tuyệt đối của các trọng số vào hàm mất mát .

- Khuyến khích trọng số thưa thớt (nhiều trọng số bằng 0).
- Hữu ích cho việc lựa chọn tính năng.

Chính quy hóa L2 (Ridge): Cộng tổng các giá trị bình phương của các trọng số vào hàm mất mát.

- Khuyến khích tập tạ nhỏ.
- Giảm tác động của các ngoại lệ.

Triển khai trong PyTorch:

- Sử dụng tham số **weight_decay** trong trình tối ưu hóa để áp dụng chính quy L2.
- Để chính quy hóa L1, bạn cần thêm hình phạt L1 theo cách thủ công vào hàm mất.

Theo dõi hiệu suất của mô hình trên tập dữ liệu xác thực để phát hiện quá mức và điều chỉnh các kỹ thuật chính quy hóa cho phù hợp.

Bài thực hành 2 - Thực hành thực hành với các kiến trúc mạng nơ-ron được đào tạo trước

Bài tập:

1) Thử nghiệm với các mô hình được đào tạo trước khác nhau (VGG, DenseNet, v.v.).

2) Điều chỉnh các siêu tham số (tốc độ học, kích thước lô, v.v.).

3) Sử dụng TensorBoard để theo dõi quá trình đào tạo.

Mục tiêu:

- Làm quen với việc tải và sử dụng các mô hình được đào tạo trước từ **torchvision.models.**
- Hiểu cách điều chỉnh và tinh chỉnh các mô hình được đào tạo trước cho các nhiệm vụ cụ thể.

Các bước:

1. Thiết lập môi trường:
   - Đảm bảo bạn đã cài đặt PyTorch và **torchvision**.
   - Nếu không, hãy cài đặt chúng bằng cách sử dụng: **pip install torch torchvision** 2. Tải Mô hình được đào tạo trước:
   - **torchvision.models** cung cấp nhiều mô hình được đào tạo trước khác nhau như ResNet, VGG, DenseNet, v.v.
   - **được huấn luyện trước=Đúng** tải trọng số được huấn luyện trước trên tập dữ liệu ImageNet.

3. Khám phá Kiến trúc mô hình:
   - In mô hình để kiểm tra cấu trúc của nó.
   - Xác định các lớp và tham số của mô hình.

4. Điều chỉnh mô hình cho một nhiệm vụ cụ thể:
   - Thông thường, lớp cuối cùng của mô hình được đào tạo trước (lớp phân loại) cần được thay thế để khớp với số lượng lớp trong tập dữ liệu của bạn.
   - Bạn có thể đóng băng các lớp trước đó để ngăn chúng cập nhật trọng số trong quá trình đào tạo lại (học chuyển giao).
   - Hoặc bạn có thể giải phóng một vài lớp cuối cùng để tinh chỉnh mô hình.

5. Chuẩn bị dữ liệu:
   - Tải và chuẩn bị tập dữ liệu của bạn.
   - Sử dụng **torchvision.transforms** để xử lý trước dữ liệu.
   - Sử dụng **torch.utils.data.DataLoader** để tạo các lô dữ liệu. 6. Đào tạo người mẫu:
   - Xác định hàm mất mát và trình tối ưu hóa.
   - Viết vòng lặp đào tạo để huấn luyện mô hình trên tập dữ liệu của bạn. 7. Đánh giá Mô hình: Đánh giá hiệu suất của mô hình trên tập dữ liệu thử nghiệm.

Example (using the CIFAR-10 dataset):

```python
import torch
    import torchvision
    import torchvision.transforms as transforms
    import torch.nn as nn
    import torch.optim as optim

    # 1. Load ResNet-18
    resnet18 = torchvision.models.resnet18(pretrained=True)

    # 2. Change the final layer
    num_ftrs = resnet18.fc.in_features
    resnet18.fc = nn.Linear(num_ftrs, 10)

    # 3. Prepare the CIFAR-10 data
    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
    ])

    trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                            download=True, transform=transform)
    trainloader = torch.utils.data.DataLoader(trainset, batch_size=4,
                                              shuffle=True, num_workers=2)

    # 4. Loss function and optimizer
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.SGD(resnet18.fc.parameters(), lr=0.001, momentum=0.9)

    # 5. Train the model
    for epoch in range(2):  # loop over the dataset multiple times

        running_loss = 0.0
        for i, data in enumerate(trainloader, 0):
            inputs, labels = data

            optimizer.zero_grad()

            outputs = resnet18(inputs)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

            running_loss += loss.item()
            if i % 2000 == 1999:    # print every 2000 mini-batches
                print('[%d, %5d] loss: %.3f' %
                      (epoch + 1, i + 1, running_loss / 2000))
                running_loss = 0.0

    print('Finished Training')
```

Hỏi đáp